

Capgemini DevOps Interview

29 Questions & Model Answers

Git · Docker · Maven · CI/CD · GCP · Kubernetes · Terraform

■ Git / Source Control

Q1. Difference between Git Fetch and Git Merge

Git

Git Fetch downloads changes from the remote repo into your local remote-tracking branch (origin/main) but does NOT apply them to your working branch. Safe to run anytime.

Git Merge takes changes from one branch and integrates them into your current branch, creating a merge commit if histories diverged.

- git fetch origin → just downloads, no change to working branch
- git merge origin/main → applies downloaded changes
- git pull = fetch + merge in one command

Interview tip: Say 'Fetch is non-destructive — it never touches your working files.'

Q2. What are the conflicts you have seen in Git?

Git

Git conflicts occur when two developers modify the same lines differently, or one deletes a file the other modified.

- Merge conflict — diverged branches edited the same line; Git marks it with <<<<<< / ===== / >>>>>>
- Rebase conflict — a commit can't replay cleanly on the new base during git rebase
- Cherry-pick conflict — a picked commit has dependencies not present in the target branch
- Delete-modify conflict — one branch deletes a file, another branch edits it

Resolution: manually edit conflict markers, then git add <file> and git commit.

Interview tip: Mention tools like VS Code merge editor or git mergetool for visual resolution.

■ Docker

Q3. Explain Docker Client Architecture

Docker

Docker uses a client-server architecture:

- Docker Client (docker CLI) — user interface, sends REST API requests to the daemon
- Docker Daemon (dockerd) — runs on host, manages images, containers, networks, volumes
- Docker Registry (Docker Hub / ECR / GCR) — stores images; daemon pulls/pushes from here

Flow: docker run nginx → client sends request to daemon → checks local image cache → pulls from registry if missing → starts container.

Client communicates with daemon via the Docker REST API over a Unix socket at /var/run/docker.sock.

Interview tip: The client and daemon can run on the same machine or the client can connect to a remote daemon via TCP.

■ Maven

Q4. What are the Maven Phases?

Maven

Maven default build lifecycle — 8 phases executed in order:

- validate — checks project is correct and all info is available
- compile — compiles source code
- test — runs unit tests
- package — packages compiled code into JAR/WAR
- verify — runs integration tests and quality checks
- install — installs package into local Maven repo (~/.m2)
- deploy — copies package to remote repo for sharing

Also: **clean** lifecycle (pre-clean → clean → post-clean) and **site** lifecycle.

Interview tip: Running mvn package automatically runs all prior phases (validate, compile, test) first.

■ CI/CD / Pipelines

Q5. What are the different types of pipelines?

CI/CD

- CI Pipeline (Continuous Integration) — triggered on code push; runs build, test, code quality scan
- CD Pipeline (Continuous Delivery) — deploys to staging automatically; production needs manual approval
- CD Pipeline (Continuous Deployment) — fully automated end-to-end including production
- Multibranch Pipeline — Jenkins auto-discovers branches; separate pipelines per branch (dev/staging/prod)
- Release Pipeline — manages versioning, changelog, artifact publishing
- Infrastructure Pipeline — applies Terraform/Ansible changes

In Jenkins: **Scripted Pipeline** (Groovy in Jenkinsfile) vs **Declarative Pipeline** (structured pipeline {} block).

Interview tip: Reference your ApnaKart Jenkins pipeline — multi-environment branch strategy (dev/test/uat/prod) with Customize overlays.

■ GCP Networking

Q6. Difference between Shared VPC and VPC Peering

GC
P

Shared VPC: A host project shares its VPC with service projects. Resources in service projects use the host's subnets. Centralized network management — one team controls networking.

VPC Peering: Two separate VPCs connect with private IP routing. Each VPC retains its own subnet and firewall management. Decentralized — both sides are equal peers. Non-transitive ($A \leftrightarrow B$ and $B \leftrightarrow C$ doesn't mean $A \leftrightarrow C$).

- Shared VPC: same org, centralized control
- VPC Peering: can be cross-org, each team manages own network

Interview tip: Shared VPC = one network admin team; VPC Peering = two independent teams that need to talk privately.

■ Kubernetes Basics

Q7. What is a DaemonSet?

Kubernetes

A **DaemonSet** ensures that one pod runs on every node (or a subset of nodes). When a new node joins, the DaemonSet controller automatically schedules the pod on it.

Common use cases:

- Log collection agents — Fluentd, Filebeat
- Node monitoring — Datadog agent, Prometheus node-exporter
- Network plugins — Calico, Weave CNI
- Storage daemons — Ceph, GlusterFS

***Interview tip:** A DaemonSet is infrastructure-level — it's for things every node needs, not app workloads.*

Q8. What are Taints and Tolerations?

Kubernetes

Taints are applied to nodes to repel pods. Format: key=value:Effect

Tolerations are applied to pods to allow scheduling on tainted nodes.

Taint Effects:

- NoSchedule — new pods without toleration won't be scheduled
- PreferNoSchedule — soft version, tries to avoid
- NoExecute — evicts existing pods that don't tolerate

Example: Taint GPU nodes so only GPU-workload pods run on them.

***Interview tip:** Taints + Tolerations control WHERE pods go; Node Affinity is the positive counterpart (attracting pods to specific nodes).*

Q9. Issues you identified in deployment times

Kubernetes

- Large Docker images causing slow pulls (no multi-stage builds)
- Sequential pipeline stages — no parallelism
- No image caching in CI — rebuilding all layers every time
- Resource requests/limits not set — pods scheduled on overloaded nodes
- Rolling update maxSurge/maxUnavailable not optimized
- Readiness probe delays configured too conservatively
- Registry in different region — network latency on pull

***Interview tip:** Reference your quantified metric — 'Reduced pipeline time from 45 min to 10 min' and explain the root cause.*

Q10. Resolutions implemented to fix deployment times

Kubernetes

- Multi-stage Docker builds → reduced image size by ~60%
- Parallelized pipeline stages (build + test simultaneously)
- Docker layer caching in Jenkins (--cache-from)
- Set proper resource requests so scheduler places pods on right nodes
- Tuned maxSurge: 2, maxUnavailable: 0 for faster zero-downtime rollouts
- Optimized readiness probe initialDelaySeconds and periodSeconds
- Regional registry (ECR in same region as EKS) for fast pulls

***Interview tip:** Always pair problem with metric: 'X was causing Y, I fixed it with Z, result was W% improvement.'*

Q11. Which command is used to describe pod status?

Kubernetes

Primary command:

```
kubectl describe pod
```

Shows: node assigned, container states, events, restart count, resource limits, conditions, volume mounts. The Events section reveals exactly why a pod is failing.

Other useful commands:

```
kubectl get pod -o wide # quick status + node + IP
```

```
kubectl logs # container logs
```

```
kubectl logs --previous # logs of crashed container
```

Interview tip: The Events section of kubectl describe pod is always your first debugging step.

Q12. What is the use of Namespace in Kubernetes?

Kubernetes

Namespaces provide **logical isolation** within a single cluster.

- Environment separation — dev, staging, prod namespaces in the same cluster
- Team isolation — team-a and team-b can't accidentally affect each other
- Resource quotas — limit CPU/memory per namespace
- RBAC scoping — grant access per namespace
- Network policies — restrict traffic between namespaces

Default namespaces: default, kube-system, kube-public, kube-node-lease.

Interview tip: Namespaces don't provide network isolation by themselves — you need NetworkPolicies for that.

■ Kubernetes Troubleshooting

Q13. Difference between CrashLoopBackOff and ImagePullBackOff

Kubernetes

CrashLoopBackOff: The container starts but immediately crashes, then Kubernetes restarts it. After repeated failures, it enters exponential backoff. The image was pulled successfully — the problem is inside the container.

ImagePullBackOff: Kubernetes cannot pull the container image. Container never starts. Causes: wrong image name/tag, private registry without credentials, network issue.

- CrashLoopBackOff → debug with `kubectl logs --previous`
- ImagePullBackOff → check image name/tag, check `imagePullSecrets`

Interview tip: CrashLoopBackOff = image pulled, app broken. ImagePullBackOff = image not found/accessible.

Q14. Reasons for CrashLoopBackOff

Kubernetes

- Application error on startup — missing config, DB connection fails, bad env variable
- OOMKilled — container exceeded memory limit, kernel killed it
- Wrong command/entrypoint — Dockerfile CMD is incorrect, process exits immediately
- Missing dependencies — required service (DB, Redis) not ready yet
- File/permission errors — app can't read config or write to path
- Liveness probe misconfigured — kills healthy container too aggressively

Debug: `kubectl logs <pod> --previous` and `kubectl describe pod <pod>` for last exit code.

Q15. Different types of Probes

Kubernetes

3 types of probes:

- Liveness Probe — Is the container alive? Fails → Kubernetes restarts it. Use for deadlocks/hangs.
- Readiness Probe — Ready to receive traffic? Fails → pod removed from Service endpoints. Use for warmup.
- Startup Probe — Did the container start successfully? Disables liveness and readiness until it passes.

Probe mechanisms: httpGet, exec (run a command), tcpSocket.

Key config: initialDelaySeconds, periodSeconds, failureThreshold, timeoutSeconds.

***Interview tip:** Readiness controls traffic routing; Liveness controls restarts. Startup protects slow-starting containers from both.*

■ Terraform

Q16. Major use of Terraform Lint (tflint)

Terraform

tflint is a static analysis tool that catches issues terraform validate misses:

- Invalid provider-specific resource configurations (e.g., unsupported EC2 instance types)
- Deprecated resource attributes
- Naming convention violations
- Unused variables and declarations
- Module best practices enforcement

Works with provider-specific rulesets (AWS, GCP, Azure plugins). Integrated into CI pipelines to catch misconfiguration before terraform apply.

***Interview tip:** tflint is to Terraform what ESLint is to JavaScript — enforces quality and catches logical errors, not just syntax.*

Q17. What is Terraform Drift?

Terraform

Drift is when real-world infrastructure differs from what Terraform state records. Happens when someone makes manual changes outside of Terraform (e.g., modifying a security group in the AWS console).

Detect drift:

```
terraform plan # shows diff between state and actual infra
terraform refresh # updates state to reflect actual infra (use carefully)
```

Risk: Next terraform apply may overwrite or destroy manual changes.

***Interview tip:** Drift is the enemy of IaC consistency — it's why we enforce Terraform-only policies in production.*

Q18. What is Terraform Taint?

Terraform

terraform taint marks a resource in state as tainted — it will be **destroyed and recreated** on next terraform apply, even if no config changed.

Use cases: resource is corrupted, misconfigured, or stuck in a bad state.

```
terraform taint aws_instance.web
```

Modern approach (Terraform 0.15.2+): taint is deprecated. Use:

```
terraform apply -replace="aws_instance.web"
```

***Interview tip:** Mention the modern replacement: terraform apply -replace. Shows you're up to date.*

Q19. What is for_each in Terraform?

Terraform

for_each iterates over a **map or set of strings** to create multiple resource instances, each with a unique key.

```
for_each = toset(["dev", "staging", "prod"])
```

Each resource is tracked by key in state: aws_s3_bucket.env["dev"], aws_s3_bucket.env["staging"].

Advantage over count: Removing one item only destroys that resource. With count, removing an index shifts all subsequent resources.

Interview tip: for_each gives stable, key-based references. Removing one item doesn't affect others — unlike count which is index-based.

Q20. What is count in Terraform?

Terraform

count creates **N identical copies** of a resource using an integer. Resources referenced by index:

aws_instance.server[0], [1], etc.

```
count = 3 # creates 3 EC2 instances
```

Use count.index inside the block for differentiation.

Limitation: Index-based. Removing element at position 1 shifts all subsequent resources, potentially causing recreation.

Interview tip: Use count for simple 'N identical things'; use for_each when each resource needs distinct identity.

Q21. Difference between for_each and count

Terraform

count: Integer-based, N identical resources indexed by number. Removing mid-list causes index shift and potential resource recreation.

for_each: Map/set-based, string keys. Removing one item only destroys that resource — others unaffected. Stable named identity in state.

- Use count when: simple repetition, resources are identical
- Use for_each when: distinct identities, dynamic maps, safer deletions

for_each cannot use a list directly — convert with toset(). count cannot use string keys.

Interview tip: In production, for_each is almost always safer than count because it avoids unintended resource destruction.

■ Kubernetes Autoscaling

Q23. Difference between HPA and VPA

Kubernetes

HPA (Horizontal Pod Autoscaler): Scales the number of pod replicas based on metrics (CPU, memory, custom). More pods at high load.

VPA (Vertical Pod Autoscaler): Adjusts CPU/memory requests and limits of existing pods. Makes pods bigger/smaller. Requires pod restart to apply new resource values.

- HPA: more pods, same size — for stateless scalable workloads (web servers, APIs)
- VPA: same pod count, bigger pods — for stateful or singleton workloads (databases, ML jobs)

Interview tip: VPA and HPA can conflict — use VPA in Recommendation mode alongside HPA to avoid clash.

■ Kubernetes / GKE Services

Q24. Different types of Services in GKE/Kubernetes

Kubernetes

- ClusterIP (default) — internal-only, accessible within the cluster only
- NodePort — exposes service on each node's IP at a static port (30000–32767)
- LoadBalancer — provisions a cloud LB (GCP/AWS/Azure), external IP assigned
- ExternalName — maps service to an external DNS name (CNAME only, no proxying)

In GKE: LoadBalancer creates an L4 TCP/UDP LB. Ingress creates an L7 HTTP(S) LB.

Interview tip: ClusterIP=internal; NodePort=dev/testing; LoadBalancer=production; ExternalName=bridging to external services.

Q25. NodePort IP/port range

Kubernetes

NodePort exposes a service on every node's IP at a static port.

Default port range: 30000–32767

Defined by --service-node-port-range flag in kube-apiserver. Customizable.

Access: NodeIP:NodePort → kube-proxy forwards to correct pod.

Three ports: port (cluster-internal), targetPort (pod/container), nodePort (external 30000–32767).

Interview tip: NodePort is rarely used in production — use LoadBalancer or Ingress instead. NodePort is good for on-prem or quick testing.

Q26. Different types of Load Balancers

Kubernetes

- L4 Load Balancer (TCP/UDP) — routes based on IP and port. Fast, low-level. Created by LoadBalancer service type.
- L7 Load Balancer (HTTP/HTTPS) — routes by URL path, host headers, cookies. Created by Ingress. SSL termination.
- Internal Load Balancer — private IP, only accessible within VPC. For microservice communication.
- External Load Balancer — public IP, internet-facing.
- Global vs Regional — GCP HTTP(S) LB is global (anycast); TCP/UDP LBs are regional.

Interview tip: L4 = faster, simpler routing; L7 = smarter routing, SSL termination, path-based rules. Ingress uses L7.

Q27. Difference between Load Balancer and Ingress. Which is best?

Kubernetes

LoadBalancer service: Creates a dedicated cloud LB per service. Each service gets its own external IP. Simple but expensive — 10 services = 10 LBs = high cost.

Ingress: Single L7 HTTP(S) LB with routing rules. One IP/LB serves many services via host/path rules. SSL termination. Cost-efficient.

- LoadBalancer: one LB per service, L4, any protocol (TCP/UDP)
- Ingress: one LB for many services, L7, HTTP/HTTPS only

Best practice: Use Ingress for HTTP/HTTPS workloads. Use LoadBalancer only for non-HTTP protocols (databases, raw TCP).

Interview tip: For HTTP services, Ingress is almost always preferred — more cost-efficient with SSL, path routing, and host-based routing.

■ GCP Serverless

Q28. Difference between Cloud Run and Cloud Functions

GCP Serverless

Cloud Run: Runs containerized applications. You bring a Docker image. Handles full HTTP request/response lifecycle. Scales to zero. Best for APIs, microservices, web apps.

Cloud Functions: Function-level execution. Write just the function code. Supports event triggers (Pub/Sub, GCS, HTTP, Firestore). Zero infrastructure management.

- Cloud Run: container → full app, longer-running requests, any language via Docker
- Cloud Functions: function → event-driven, short execution, native GCP event triggers

***Interview tip:** Cloud Run when you need custom runtime/containers; Cloud Functions for quick GCP event-driven triggers with minimal code.*

■ GCP Storage

Q29. Different types of Buckets in Google Cloud Storage

GCP Storage

GCS buckets use **storage classes** — cost-vs-access-frequency tradeoff:

- Standard — frequent access, highest cost, best availability. Hot data, content serving.
- Nearline — accessed ~once/month. Lower storage cost, retrieval fee. Backups.
- Coldline — accessed ~once/quarter. Even lower cost, higher retrieval fee. Disaster recovery.
- Archive — accessed less than once/year. Cheapest storage, highest retrieval cost. Long-term archival.

Bucket location types: **Regional**, **Dual-Region**, or **Multi-Region** for availability.

***Interview tip:** Autoclass feature can automatically move objects between storage classes based on access patterns.*